Exercises_with_solutions

December 26, 2024

1 MN5813: Advanced Data Visualisation (Exercises with Solutions)

This notebook provides exercises for advanced data visualisations using Pandas. Exercises are designed to be completed in approximately 90 minutes by students who have little familiarity with the topics.

Note: This Jupyter Notebook was originally compiled by Alex Reppel (AR) based on conversations with ClaudeAI *(version 3.5 Sonnet)*.

1.1 Introduction

1.1.1 Overview

1. Exercise 1: Customer segmentation analysis
2. Exercise 2: Improved segment visualiation
3. Exercise 3: Marketing effectiveness analysis
4. Exercise 4: Improve visual appeal *(of the previous visualisation)*

1.1.2 Tips

- Review the demonstration notebook for examples and syntax
- Pay attention to plot customisation options
- Consider the best way to present the data clearly
- Don't forget to add proper labels and titles
- Use appropriate color schemes

Remember: The goal is to create clear, informative visualizations that effectively communicate the data's story.

1.2 Setup

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[2]:
```
# Set the style
plt.style.use("seaborn-v0_8-muted")
```

[3]:
```python
# This time, we're reading in two different DataFrames
business_df = pd.read_csv("assets/business_metrics.csv")
retail_df = pd.read_csv("assets/retail_sales.csv")
```

[4]:
```python
business_df.head(3)
```

[4]:
```
         date  visitors  marketing_spend  conversion_rate  avg_order_value  \
0  2023-06-01      1049           517.89           0.0195            53.78
1  2023-06-02      1234           528.04           0.0226            48.94
2  2023-06-03      1414           554.15           0.0245            59.65

   revenue  satisfaction
0  1101.04          4.36
1  1361.81          3.93
2  2065.86          3.92
```

[5]:
```python
retail_df.head(3)
```

[5]:
```
   transaction_id                 date  customer_id customer_segment  \
0               1  2023-06-01 01:00:00          365          Regular
1               2  2023-06-01 01:00:00          300          Regular
2               3  2023-06-01 02:00:00            7          Regular

  product_category  quantity  unit_price  discount  discounted_price  \
0            Books         3       21.50       0.0             21.50
1    Home & Garden         2       56.60       0.2             45.28
2            Books         1       40.22       0.0             40.22

   total_amount payment_method  satisfaction_score
0         64.50    Credit Card                4.25
1         90.56           Cash                4.79
2         40.22    Credit Card                4.35
```

1.3 Exercise 1: Basic customer segmentation analysis

1.3.1 Challenge

Create a `lineplot` *(using* ***`seaborn`****)* comparing average transaction values across segments. Try to create a function called `basic_segment_analysis` that takes `retail_df` as input and returns a `plt` object. *(Note: This visualisation has limitations we'll address in Exercise 2.)*

1.3.2 Requirements

1. Create a line plot showing average transaction value per customer segment over time
2. Add 95% confidence intervals using seaborn's capabilities
3. Format y-axis to show currency *(£)*
4. Include clear legend and proper title
5. Add gridlines with 30% opacity

1.3.3 Useful matplotlib/seaborn functions

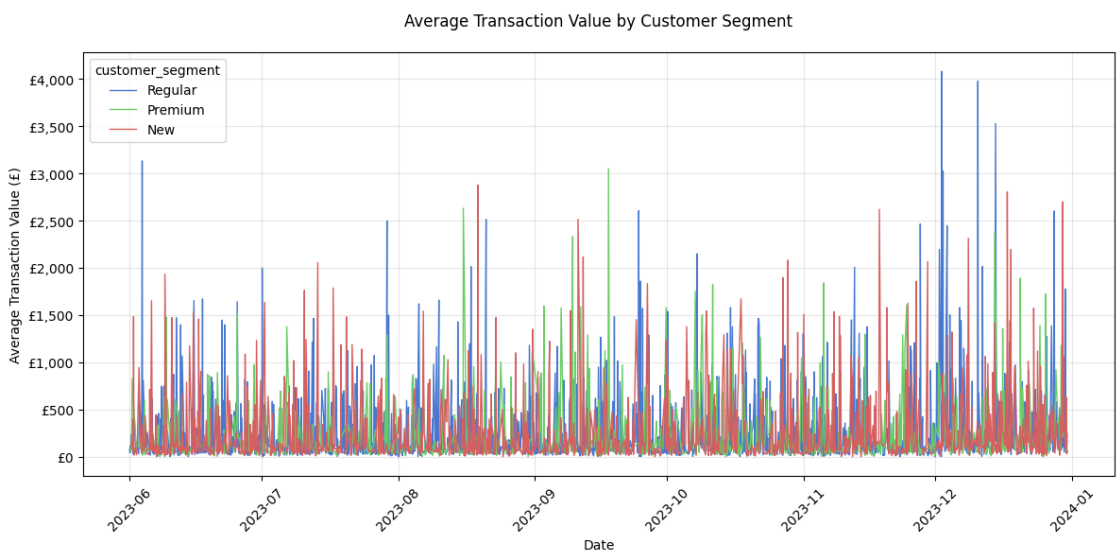
1. `plt.figure()`: Creates new figure window
2. `sns.lineplot()`: Creates line plot *(with optional confidence intervals)*
3. `plt.gca()`: Gets current axes *(subplot)*
4. `plt.gcf()`: Gets current figure *(entire plot)*
5. `plt.tight_layout()`: Automatically adjusts subplot spacing

1.3.4 Code

[6]:
```
def basic_segment_analysis(df):
    """Returns a line plot comparing average transaction values across segments.

    Requires a `df` object.
    """

    # Convert date column to datetime format for proper time series plotting
    df["date"] = pd.to_datetime(df["date"])

    # Calculate daily averages per customer segment
    # Groups data by date and segment, then calculates mean transaction value
    daily_avg = df.groupby(["date", "customer_segment"])["total_amount"].mean().
↪reset_index()

    # Create a new figure with specified size
    plt.figure(figsize=(12, 6))

    # Create line plot with confidence intervals
    # x: date values
    # y: total_amount (average transaction value)
    # hue: creates separate lines for each customer segment
    # ci=95: adds 95% confidence interval bands around each line
    sns.lineplot(
        data=daily_avg,
        x="date",
        y="total_amount",
        hue="customer_segment",
        errorbar=("ci", 95),
        linewidth=1
    )

    # Add title and labels
    plt.title("Average Transaction Value by Customer Segment", pad=20)
    plt.xlabel("Date")
    plt.ylabel("Average Transaction Value (£)")

    # Format y-axis to show currency values with £ symbol
    # lambda function converts numbers to currency format
```

```
    plt.gca().yaxis.set_major_formatter(
        plt.FuncFormatter(lambda x, p: f"£{x:,.0f}"))

    # Add subtle grid in background
    plt.grid(True, alpha=0.3)

    # Rotate date labels for better readability
    plt.xticks(rotation=45)

    # Adjust spacing to prevent label cutoff
    plt.tight_layout()

    # Get current figure and return it
    # plt.gcf() means "get current figure"
    # This allows the function to return the plot
    # for further modification if needed
    return plt.gcf()
```

[7]:
```
# Usage example
figure = basic_segment_analysis(retail_df)  # Using retail_df!
plt.show()
```



1.4 Exercise 2: Improved segment visualiation

1.4.1 Challenge

Create a clearer view using small multiples across `customer_segments`:

1. Regular
2. Premium

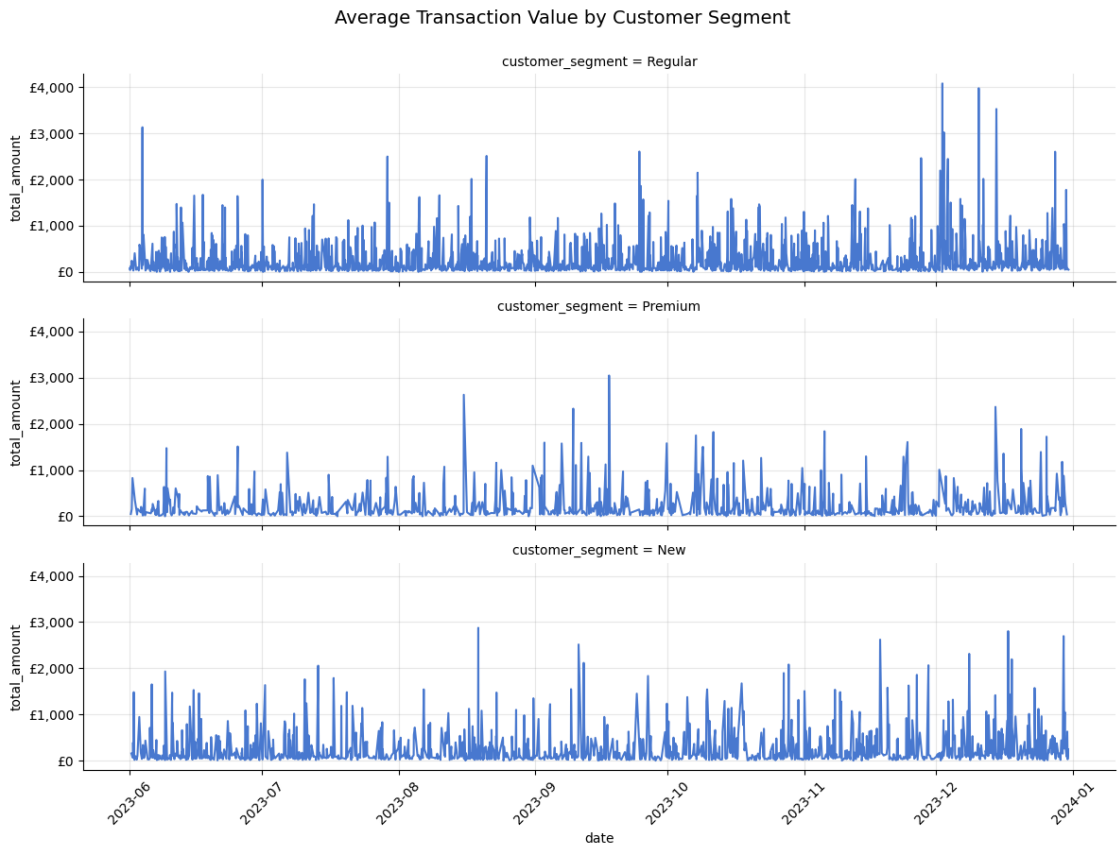
3. New

(Note: This addresses the overlapping issues from Exercise 1.)

1.4.2 Code

[8]:
```
def improved_segment_analysis(df):

    df["date"] = pd.to_datetime(df["date"])
    daily_avg = df.groupby(["date", "customer_segment"])["total_amount"].mean().
 ↪reset_index()

    g = sns.FacetGrid(
        daily_avg,
        col="customer_segment",
        col_wrap=1,
        height=3,
        aspect=4)

    g.map_dataframe(
        sns.lineplot,
        x="date",
        y="total_amount",
        errorbar=("ci", 95))

    for ax in g.axes.flat:
        ax.grid(True, alpha=0.3)
        ax.tick_params(axis="x", rotation=45)
        ax.yaxis.set_major_formatter(
            plt.FuncFormatter(lambda x, p: f"£{x:,.0f}"))

    g.fig.suptitle(
        "Average Transaction Value by Customer Segment",
        y=1.0,
        fontsize=14)

    plt.tight_layout()
    return g.fig
```

[9]:
```
improved_figure = improved_segment_analysis(retail_df)  # Again, using␣
 ↪retail_df!
plt.show()
```



1.5 Exercise 3: Marketing effectiveness analysis

1.5.1 Challenge

Create a dual-axis plot analyzing marketing effectiveness by combining data from both datasets. Create a function called `analyse_marketing_effectiveness` that takes `business_df` and `retail_df` as inputs and returns a `plt` object.

1.5.2 Requirements

1. Combine daily conversion rates with customer satisfaction scores
2. Add 7-day moving averages for both metrics
3. Use dual y-axes *(conversion rate % and satisfaction score)*
4. Include clear legend *(underneath the main chart!)* and proper formatting
5. Add gridlines with 30% opacity

1.5.3 Useful matplotlib/seaborn functions

1. `plt.subplots()`: Creates figure and axis objects
2. `ax.twinx()`: Creates secondary y-axis
3. `ax.plot()`: Creates line plot on specified axis
4. `ax.bar()`: Creates bar plot

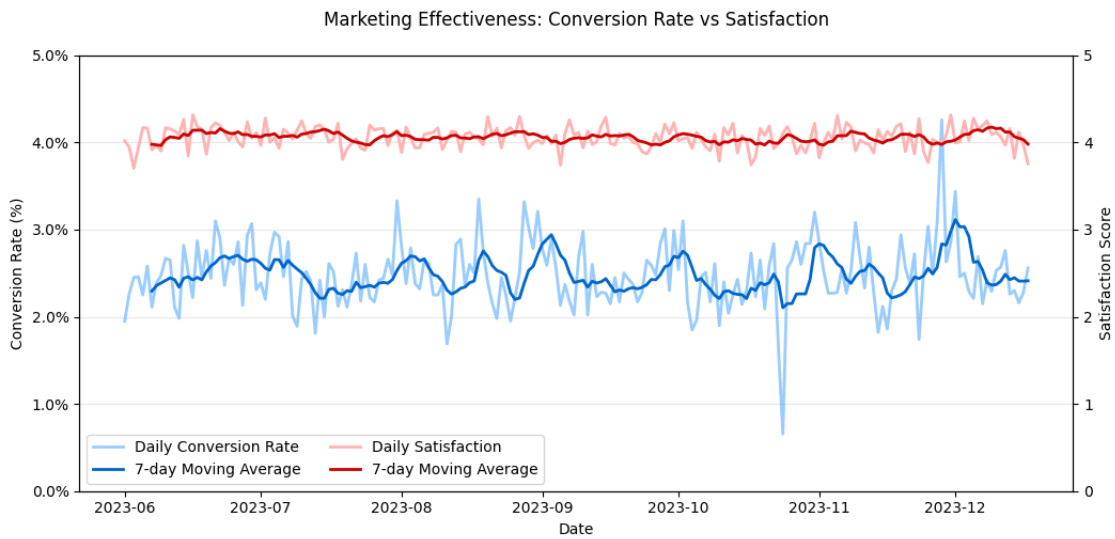
5. `pd.rolling()`: Calculates moving averages
6. `pd.merge()`: Combines two `DataFrames`

1.5.4 Code

[10]:
```
def analyse_marketing_effectiveness(business_df, retail_df):

    # Data preparation
    retail_df["date"] = pd.to_datetime(retail_df["date"]).dt.date
    daily_satisfaction = retail_df.groupby("date")["satisfaction_score"].mean().
↪reset_index()
    business_df["date"] = pd.to_datetime(business_df["date"]).dt.date
    combined_df = pd.merge(business_df, daily_satisfaction, on="date",␣
↪how="inner")

    # Calculate moving averages
    combined_df["satisfaction_ma"] = combined_df["satisfaction_score"].
↪rolling(7).mean()
    combined_df["conversion_ma"] = combined_df["conversion_rate"].rolling(7).
↪mean()

    # Create figure with two y-axes
    fig, ax1 = plt.subplots(figsize=(12, 6))
    ax2 = ax1.twinx()

    # Define colors - light for daily, dark for average
    conv_light = "#99ccff"  # Light blue
    conv_dark = "#0066cc"   # Dark blue
    sat_light = "#ffb3b3"   # Light red
    sat_dark = "#cc0000"    # Dark red
    spend_color = "#95a5a6" # Grey for marketing spend

    # Conversion rate plots
    line1 = ax1.plot(
        combined_df["date"],
        combined_df["conversion_rate"] * 100,
        color=conv_light,
        linewidth=2,
        label="Daily Conversion Rate")
    line1_ma = ax1.plot(
        combined_df["date"],
        combined_df["conversion_ma"] * 100,
        color=conv_dark,
        linewidth=2,
        linestyle="-",
        label="7-day Moving Average")
```

```
    # Set conversion rate limits
    ax1.set_ylim(0, 5)  # Manually added: Min/max for conversation rate

    # Satisfaction plots
    line2 = ax2.plot(
        combined_df["date"],
        combined_df["satisfaction_score"],
        color=sat_light,
        linewidth=2,
        label="Daily Satisfaction")

    line2_ma = ax2.plot(
        combined_df["date"],
        combined_df["satisfaction_ma"],
        color=sat_dark,
        linewidth=2,
        linestyle="-",
        label="7-day Moving Average")

    # Set satisfaction limits
    ax2.set_ylim(0, 5)

    # Customize axes
    ax1.set_xlabel("Date")
    ax1.set_ylabel("Conversion Rate (%)")
    ax2.set_ylabel("Satisfaction Score")

    # Format conversion rate as percentage
    ax1.yaxis.set_major_formatter(
        plt.FuncFormatter(lambda x, p: f"{x:.1f}%"))

    # Add legend below the chart
    lines = line1 + line1_ma + line2 + line2_ma
    labels = [l.get_label() for l in lines]
    ax1.legend(lines, labels, loc="lower left", ncol=2)

    plt.title("Marketing Effectiveness: Conversion Rate vs Satisfaction",␣
 ↪pad=20)
    plt.grid(True, alpha=0.3)
    plt.xticks(rotation=45)

    plt.subplots_adjust(bottom=0.2)

    return fig
```

[11]:
```
figure = analyse_marketing_effectiveness(business_df, retail_df)
plt.show()
```



1.6 Exercise 4: Improve visual appeal

1.6.1 Challenge

Enhance the visual appeal of the marketing effectiveness dual-axis plot by creating a function called `analyse_marketing_effectiveness_enhanced` that takes `business_df` and `retail_df` as inputs and returns a `plt` object with improved aesthetics.

1.6.2 Requirements

1. Use a professional color scheme
   - Steel blue shades for conversion rates *(light: #a8d5ff, dark: #2c5282)*
   - Coral shades for satisfaction scores *(light: #fed7d7, dark: #9b2c2c)*
   - Subtle grey background *(#f8fafc)*
2. Enhance data presentation
   - Make daily data subtle *(alpha = 0.6)*
   - Emphasize moving averages with thicker lines
   - Set y-axes limits from 0 to 5 for both metrics
   - Format month and year with line break
   - Show conversion rate percentages without decimals
3. Style the typography
   - Two-line title with bold first line only
   - Colored y-axis labels matching their data
   - No x-axis label
4. Add professional finishing touches
   - Dashed grid lines *(alpha = 0.2)*
   - Two-column legend below the chart
   - Proper spacing between all elements

1.6.3 Useful matplotlib/seaborn functions

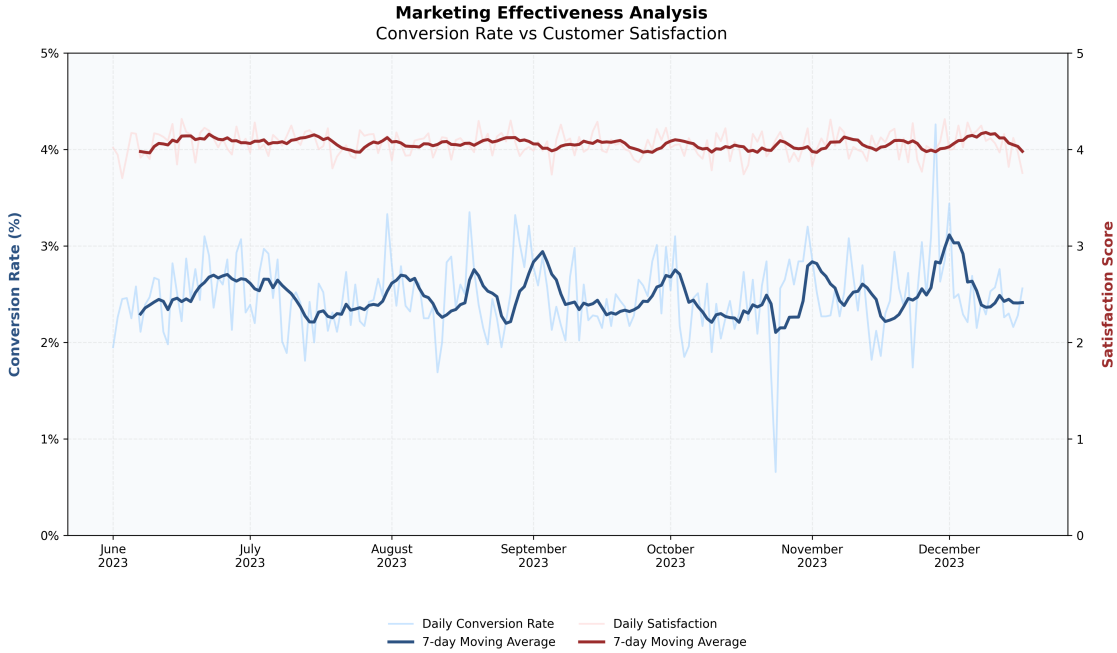
1. `plt.style.use("seaborn-v0_8-muted")`: Sets the overall style
2. `ax.twinx()`: Creates secondary y-axis
3. `mdates.DateFormatter("%B\n%Y")`: Formats dates
4. `plt.LinearLocator(6)`: Sets consistent tick marks
5. `fig.text()`: Adds text at specific coordinates
6. `os.makedirs()`: Creates directories for saving

1.6.4 Hints

1. Use `fig.text()` instead of `plt.title()` for multi-style titles
2. Match y-axis label colors to their respective data colors
3. Set both y-axes to the same range *(0-5)* for better comparison
4. Use the `seaborn-v0_8-muted` style for a professional look
5. Create the output directory before saving the figure
6. Remember to handle the figure margins to prevent cutoff

1.6.5 Example

This is an example output of the desired result. Try to stay as close as possible to it.



1.6.6 Code

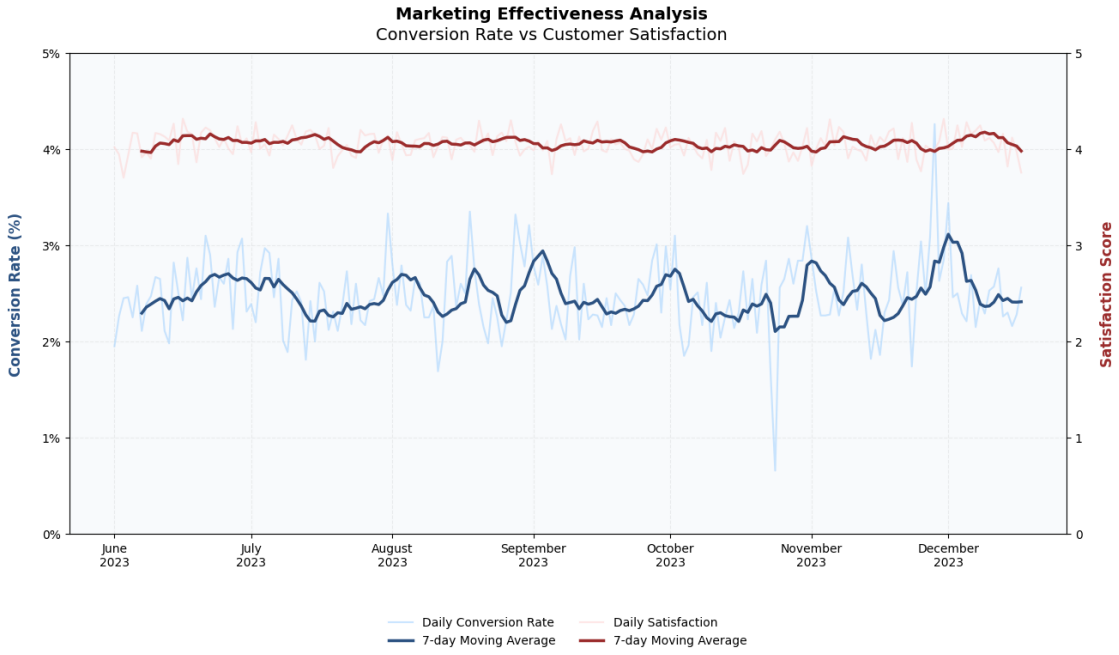
[12]:
```
# Necessar to convert months from numbers to text
import matplotlib.dates as mdates


def analyse_marketing_effectiveness_enhanced(business_df, retail_df):
```

```
# Data preparation (same as before)
retail_df["date"] = pd.to_datetime(retail_df["date"]).dt.date
daily_satisfaction = retail_df.groupby("date")["satisfaction_score"].mean().
↪reset_index()
business_df["date"] = pd.to_datetime(business_df["date"]).dt.date
combined_df = pd.merge(business_df, daily_satisfaction, on="date",␣
↪how="inner")

# Calculate moving averages
combined_df["satisfaction_ma"] = combined_df["satisfaction_score"].
↪rolling(7).mean()
combined_df["conversion_ma"] = combined_df["conversion_rate"].rolling(7).
↪mean()

# Set style for better visualization
plt.style.use("seaborn-v0_8-muted")

# Create figure with two y-axes and larger size
fig, ax1 = plt.subplots(figsize=(15, 8))
ax2 = ax1.twinx()

# Enhanced color scheme using more professional colors
conv_light = "#a8d5ff"    # Lighter steel blue
conv_dark = "#2c5282"     # Dark steel blue
sat_light = "#fed7d7"     # Lighter coral
sat_dark = "#9b2c2c"      # Dark coral

# Add subtle background shading for visual depth
ax1.set_facecolor("#f8fafc")
fig.patch.set_facecolor("#ffffff")

# Conversion rate plots with enhanced styling
line1 = ax1.plot(
    combined_df["date"],
    combined_df["conversion_rate"] * 100,
    color=conv_light,
    linewidth=1.5,
    alpha=0.6,

    label="Daily Conversion Rate")

line1_ma = ax1.plot(
    combined_df["date"],
    combined_df["conversion_ma"] * 100,
    color=conv_dark,
    linewidth=2.5,
```

```
        label="7-day Moving Average")

    # Set conversion rate limits to match satisfaction scale
    # Again, this only makes sense as we know that there's no value > 5
    ax1.set_ylim(0, 5)

    # Satisfaction plots with enhanced styling
    line2 = ax2.plot(
        combined_df["date"],
        combined_df["satisfaction_score"],
        color=sat_light,
        linewidth=1.5,
        alpha=0.6,
        label="Daily Satisfaction")

    line2_ma = ax2.plot(
        combined_df["date"],
        combined_df["satisfaction_ma"],
        color=sat_dark,
        linewidth=2.5,
        label="7-day Moving Average")

    # Set satisfaction limits explicitly (0 = min, 5 = max)
    ax2.set_ylim(0, 5)

    # Set the same number of ticks for both axes
    ax1.yaxis.set_major_locator(plt.LinearLocator(6))  # 6 ticks from 0 to 5
    ax2.yaxis.set_major_locator(plt.LinearLocator(6))  # 6 ticks from 0 to 5

    # Enhanced axes styling with matching colors
    ax1.set_xlabel("")  # We don't need 'Date' here as tick labels are self␣
↪explanatory
    ax1.set_ylabel(
        "Conversion Rate (%)",
        fontsize=12,
        fontweight="bold",
        labelpad=15,
        color=conv_dark)
    ax2.set_ylabel(
        "Satisfaction Score",
        fontsize=12,
        fontweight="bold",
        labelpad=15,
        color=sat_dark)

    # Format conversion rate as percentage with no decimal points
    ax1.yaxis.set_major_formatter(
```

```
        plt.FuncFormatter(lambda x, p: f"{int(x)}%"))

    # Customize grid with 0.2 alpha
    ax1.grid(True, linestyle="--", alpha=0.2)

    # Format x-axis dates
    ax1.xaxis.set_major_locator(mdates.MonthLocator())
    ax1.xaxis.set_major_formatter(mdates.DateFormatter("%B\n%Y"))

    # Add legend with enhanced styling
    lines = line1 + line1_ma + line2 + line2_ma
    labels = [l.get_label() for l in lines]
    leg = ax1.legend(
        lines,
        labels,
        loc="upper center",
        bbox_to_anchor=(0.5, -0.15),
        ncol=2,
        frameon=False,
        fontsize=10)

    # Add title with enhanced styling - first line bold, second line normal
    fig.text(0.5, 0.95, "Marketing Effectiveness Analysis",
            fontsize=14, fontweight="bold", ha="center")
    fig.text(0.5, 0.92, "Conversion Rate vs Customer Satisfaction",
            fontsize=14, fontweight="normal", ha="center")

    # Format tick labels
    ax1.tick_params(axis="both", labelsize=10)
    ax2.tick_params(axis="both", labelsize=10)

    # Adjust layout to prevent label cutoff
    plt.subplots_adjust(bottom=0.2, top=0.9)

    # Add a subtle border around the plot
    for spine in ax1.spines.values():
        spine.set_edgecolor("#dedede")
        spine.set_linewidth(0.8)

    return fig
```

[13]:
```
figure = analyse_marketing_effectiveness_enhanced(
    business_df,
    retail_df)
plt.show()
```



Save output as a `.png` file:

[14]:
```
import os

# Create directory if it doesn't exist
os.makedirs("assets/examples", exist_ok=True)
# Save figure to the 'assets/' folder
figure.savefig(
    "assets/examples/4.png",
    dpi=300,
    bbox_inches="tight")
```